

Aaliyah Phillips

10/9/2026

Reflection journal

For midterms, our team voted to use the Telco Customer Churn dataset to predict whether a customer will leave the telecom company. In my respective group I was tasked with categorical encoding, one-hot encoding, and feature engineering. Before working on my part of the project, I revisited previous labs in order to freshen my memory on the techniques, focusing specifically on Module 5.

Now going into the project, I understood that it was my responsibility to transform the raw dataset into a clean and numerical format ready for training. The Telco dataset possessed 20 columns and 7043 lines of data. Since real world data contains a mixture of object types, missing fields, etc. the machine algorithm would not be able to process it as is. I start by making a copy of train data to work on. Then I addressed the missing values in TotalCharges identified in my teammate's EDA. To fix this I used pandas to coerce blanks into numerical formats, filling in missing entries with 0. Next, in order to I engineered several domain-specific features: ChargesPerMonth, TenureGroup, LongTermCustomer(as a binary flag for accounts active over two years 0, and SupportServiceCount(to quantify account stickiness across add-on services like OnlineSecurity and TechSupport).

Following feature extraction, I mapped binary variables (Churn, Partner, Dependents, PhoneService, PaperlessBilling, and gender) into zero-or-one numeric formats, and encoded Contract ordinally (Month-to-month: 0, One year: 1, Two year: 2). For nominal features without ordering (MultipleLines, InternetService, PaymentMethod, and TenureGroup), I executed one-hot encoding using `pd.get_dummies` with `drop_first=True`. Though I did not learn this in class, I used `get_dummies` into binary features to avoid the "Dummy Variable Trap", using K-1 binary columns to represent K categories, this is to avoid redundant data. Finally, I validated the transformation by plotting the distributions of raw versus ordinal contract variables and verifying that zero non-numeric columns remained.

Completing this area of the project has been the most challenging assignment I've done so far this semester. I often had to revise my code, revisit past modules and labs, or rely on outside sources discussing encoding in order to complete the steps in my work. It was a reminder for me that while studying ML concepts are important. applying what I learned in class is just as vital and not to solely rely on self-study.

Seth Alvarez

Professor Rao

ITAI 1371

September 9, 2026

Midterm Reflection Journal

I worked on the Exploratory Data Analysis on the training data for our Telco churn dataset. My goal was to understand the data before anyone cleaned or changed it.

My takeaway is that EDA decides what the later steps should do. For example, TotalCharges had 7 missing values, and I would have filled them with the median until I saw that all 7 customers had a tenure of 0. They had no bill yet, so the right value was 0, which matches what Module 4 says about asking why data is missing before picking a solution. I also noticed that churn is 73.5% No and 26.5% Yes, so a model could be right 73.5% of the time by always guessing No. That made me realize why accuracy alone isn't enough to tell if a model is good, and why the classes need balancing. I'll use the EDA in the Final since the patterns I found showed which features, such as contract type and payment method, should matter most for the models.

Reflective Journal KeiCharaFleeks

I expected missing data to be obvious, but it doesn't always show up as NaN. In some versions of this dataset, a blank TotalCharges is stored as a single space. If I had relied only on `isnull()`, it would have reported zero missing values and I would have moved on with bad data. That changed how I think about checking data. I look at the data in more than one way and print the rows themselves before deciding something is clean. The hardest decision was the 9 duplicate rows. Without a customer ID, I couldn't prove they were the same customer entered twice. They could have been two different people with identical answers. I removed them because exact copies would get extra weight in training, but I wasn't fully comfortable deleting data I couldn't confirm was redundant. The missing values taught me something similar. A mean or median fill would have been the default move, and it would have given brand-new customers charges they never paid. Seeing that all 7 had tenure = 0 made 0 the obvious answer. I learned that understanding what the data means matters more than knowing which function to call. My section sat in the middle of the notebook, so it had to fit what came before and after it. Seth's EDA gave me a clear list of problems, which made my job easier. I wanted Aaliyah and Elquin to be able to run their sections without changing their code, so I replaced `train_data` with the cleaned version instead of making them switch to a new variable name.

Reflection ElquinPonce

For my part of the midterm I worked on feature scaling and normalization on the training data. I used StandardScaler on tenure, MonthlyCharges, and TotalCharges to get them to a mean of 0 and standard deviation of 1, and I also applied MinMaxScaler separately to normalize the same columns between 0 and 1. I made before and after histograms for each column so you can actually see the shift in distribution once scaling was applied. This step matters because models like logistic regression or KNN can get thrown off if one feature has a much bigger range than another, so putting everything on the same scale keeps the model from treating a feature like TotalCharges as more important just because its numbers are bigger. The class balancing part of my section is still pending since it depends on the categorical columns being encoded first, so once that's ready I'll run SMOTE on the Churn target to fix the imbalance between customers who stayed and customers who left.

Diane Lenhoff Midterm Project — Reflective Journal

This project helped me better understand how the different steps of preparing data for machine learning work together. I researched three possible datasets and worked with my team to select one that met the professor's requirements. I learned that choosing a dataset requires enough data, a clear prediction goal, and information that can be prepared for machine learning.

I completed the **70/30 train/test split using Python**, which helped me understand why training and testing data must remain separate. I also learned how important the order of preprocessing is when working as a team. Each person completed a different section, but the sections had to work together and use the correct version of the data.

One area I learned more about was **class balancing**. I learned that random oversampling duplicates records from the smaller Churn group to create equal groups rather than changing customers from one class to another. I also learned why balancing should only be performed on the training data.

I gained more experience using **Google Colab and GitHub**, including troubleshooting code, combining team members' work, managing files, and organizing the final project.

Overall, this project gave me a better understanding of dataset selection, train/test splitting, cleaning, encoding, feature engineering, class balancing, scaling, normalization, and teamwork. My biggest lesson was that each preprocessing step is connected, and the data must move through the steps in the correct order.